

Plano de Consenso — Estado Ideal da App

Manutenção — Estádio Municipal de Leiria · PWA offline-first
Consolidação de duas revisões técnicas independentes · 17 de agosto de 2026 · Commit `b4d6e1f`
Todos os números foram **re-verificados por execução real** antes de entrar neste documento.

1. Onde as Duas Revisões Concordam

Duas análises independentes chegaram às mesmas conclusões nos pontos que importam. Quando duas revisões que não se falaram apontam para o mesmo sítio, isso deixa de ser opinião e passa a ser diagnóstico.

Achado	Rev. A	Rev. B	Consenso verificado
Compressão de fotos ausente	Sim	Sim (crítico)	CONFIRMADO — <code>main.js:832</code> guarda o ficheiro original. O <code>PROJECT.md</code> promete 1280px/180 KB. Não existe.
Ghost code: <code>stadiumMap.js</code>	321 linhas	15,4 KB	CONFIRMADO — 16 KB, zero importações. Ambos certos.
Função <code>esc()</code> duplicada	16×	12×	CONFIRMADO — contagem real: 16 ficheiros , em 3 variantes diferentes.
Estilos inline no JS	220	216	CONFIRMADO — 220 no <code>src/</code> ; 216 se excluir o <code>pdfService</code> . Mesma dívida.
<code>main.js</code> monolítico	1 176 linhas	1 176 linhas / 46 KB	CONFIRMADO — 48 KB reais.
<code>confirm()</code> nativo	8 usos	Visualmente pobre	CONFIRMADO — 8 usos.
<code>components.css</code> gigante	3 079 linhas	70 KB	CONFIRMADO — 72 KB reais.
Ecrã Hoje sobrecarregado	2 ações a competir	Scroll excessivo	CONFIRMADO — mesma causa, ângulos diferentes.
QR nos equipamentos é a falta nº 1	Prioridade máxima	Recomendação 8.1	ACORDO TOTAL — coluna <code>qr_code</code> já existe na base de dados.
Calendário de jogos	Crítico	Recomendação 8.3	ACORDO TOTAL
Histórico por equipamento	Decide orçamentos	Recomendação 8.5	ACORDO TOTAL
As bases estão bem feitas	8,5/10 dados	"acima da média"	ACORDO TOTAL — 120 testes passam.

2. Onde Discordam — e Quem Tem Razão

Corri os comandos. Estes são os pontos onde as revisões divergem, e o que os factos dizem.

Ponto	Divergência	Verificação real
-------	-------------	------------------

dashboard.js	B diz "verificar se ainda é referenciado antes de eliminar". A diz "rota inalcançável".	A está mais perto. É importado (main.js:7) e instanciado (main.js:290) — <i>não</i> é ghost code. Mas a rota <code>metrics</code> não tem botão nenhum na app que lá chegue . Código vivo, ecrã inalcançável. Não apagar: ligar.
Retry / backoff	B elogia: "retry com backoff exponencial". A alerta: " <code>retryCount</code> nunca é lido".	A está certa. Existem dois conceitos: o backoff <i>do servidor</i> (existe, <code>syncEngine.js:18</code> , bem feito) e o <i>retry por item</i> (não existe — <code>retryCount</code> é escrito a 0 em 12 sítios e lido em zero). B elogiou uma coisa que não existe. A fila pode crescer sem fim.
Testes	B: 114 unitários, 34 E2E.	Números atualizados: 120 unitários (executados agora) e 34 E2E (20+7+5+2). B estava certa nos E2E, desatualizada nos unitários.
Linhas de código	B: 13 700 linhas / 518 KB.	15 802 linhas contadas em 41 ficheiros de código. A diferença é o CSS, que B pode ter excluído.
Indicador "Sincronizado"	B não menciona. A diz que mente.	A está certa e é grave. <code>markSynced</code> nunca é chamado pelo <code>syncEngine</code> . Todos os cartões dizem "Local" para sempre. O técnico não sabe se o trabalho chegou.
<code>locationId</code> nulo	B não menciona. A diz que perde trabalho.	A está certa. <code>quickCapture.js:220</code> passa <code>null</code> ; <code>reportsRepo.js:108</code> lança erro. Escrever o local à mão perde o registo.
IDs de HTML duplicados	B não menciona.	Confirmado por grep. <code>qc-description</code> , <code>qc-loc-search</code> , <code>qc-loc-dropdown</code> e <code>btn-save-capture</code> existem em dois ficheiros ao mesmo tempo.
Fugas de memória em dropdowns	B classifica prioridade "Baixa".	Subir para Alta. O ecrã Hoje redesenha a cada gravação e acumula um ouvinte de cliques em cada uma. Num turno de 8 horas com 30 registos, são 30 ouvintes acumulados. Um técnico não fecha a app.

Método usado para resolver desacordos

Regra simples: quando as duas revisões discordam, corre-se o comando. Nenhuma afirmação entrou neste plano por concordância entre revisões — só por verificação. Cinco afirmações da revisão B foram corrigidas por este processo, e duas da revisão A foram refinadas.

3. O Que a Revisão B Trouxe de Novo — e Vale a Pena

A segunda revisão encontrou seis coisas que a primeira não viu. Verifiquei todas. Estas entram no plano:

Achado novo	Verificado	Porque entra
Sem <i>debounce</i> na pesquisa	Sim	<code>history.js:213</code> redesenha a lista inteira a cada tecla. Com 2 000 avarias, escrever "relvado" são 7 redesenhos completos. Correção de 15 minutos, efeito imediato.
<code>backdrop-filter: blur(12px)</code> no cabeçalho	Sim	<code>theme.css:331</code> . Força repintura a cada rolagem. Num Android antigo — que é o telemóvel provável do técnico — dá arrastos visíveis. Substituir por cor sólida a 96 %: 5 minutos.

Sem lazy loading de imagens	Sim	Zero usos de <code>IntersectionObserver</code> ou <code>loading="lazy"</code> . Com a compressão feita, isto passa de crítico a desejável — mas continua a valer.
Sem haptic feedback	Sim	Zero usos de <code>navigator.vibrate</code> . Excelente observação contextual: ao lado de um gerador a trabalhar, o técnico não ouve nem lê a mensagem de sucesso — mas sente a vibração. 30 minutos de trabalho, valor real.
Sem undo ao eliminar	Sim	Eliminar é definitivo. Com luvas, tocar no botão errado acontece. Snackbar com "Desfazer" durante 5 s resolve.
Modo turno / passagem de testemunho	—	A melhor ideia funcional das duas revisões. Um estádio tem turnos. "O projetor da torre norte faz um barulho estranho" é informação que hoje se perde entre técnicos. Ver secção 6.
Templates de avaria e checklists	—	Casa perfeitamente com o QR: ler etiqueta do projetor → template "Manutenção de projetores" já preenchido. As duas ideias multiplicam-se.
Dropdown position:absolute no iOS	Plausível	Não consigo confirmar sem um iPhone real. Fica na lista para testar em dispositivo , não para corrigir às cegas.

O que da revisão B eu recomendo *não* fazer

Sugestão	Porque não, ou não agora
Migrar para TypeScript	Semanas de trabalho, zero valor visível para o técnico. Uma app de 15 mil linhas com 120 testes já tem rede de segurança. Se um dia entrar, entra ficheiro a ficheiro via JSDoc + <code>checkJs</code> — sem reescrever nada.
Virtualização de listas (>50 itens)	Resolve o sintoma errado. O problema não é o número de cartões, são as fotos guardadas dentro de cada registo . Separar as fotos (item 19) torna a virtualização desnecessária.
Machine learning para prever avarias	Precisa de anos de dados limpos e de custos por intervenção que ainda não são registados. Antes disto: registar quem, quanto e em que equipamento. Aí a conversa faz sentido.
Sentry / analytics de produção	Um estádio, um a três técnicos. Um botão "Enviar diagnóstico" nas Definições dá o mesmo resultado sem dependências externas nem dados a sair da rede municipal.
Preconnect a domínios externos	Não se aplica: a app é 100 % offline por desenho. Sem fontes externas, sem CDN. Isso é uma <i>qualidade</i> , não uma falta.

4. O Plano Único — Cinco Vagas

Este é o plano consolidado. Uma só lista, ordenada por **valor a dividir pelo esforço**, com a origem de cada item marcada. Substitui os dois roteiros anteriores.

Vaga 0 — A Manhã que Muda Tudo (4 horas)

Cinco correções, quatro horas, risco praticamente zero. Duas delas param perda de dados e uma faz a app deixar de mentir ao técnico. **Se só fizeres isto, já valeu.**

#	O quê	Tempo	Fonte	Efeito
1	Corrigir <code>locationId</code> nulo <code>quickCapture.js:220</code> → <code> 'LOC_UNKNOWN'</code>	30 min	A	Deixa de perder o registo quando o local é escrito à mão
2	Marcar como sincronizado após envio chamar <code>markSynced</code> no <code>syncEngine.js:236</code>	1 h	A	O indicador deixa de mentir; o técnico passa a saber que o trabalho subiu
3	Apagar ghost code <code>stadiumMap.js</code> , <code>audioService.js</code> , <code>card.classList.remove(i.cls)</code>	20 min	A B	–500 linhas, risco zero (zero importações)
4	Reativar o zoom tirar <code>user-scalable=no</code> do <code>index.html:5</code>	5 min	A	Ampliar para ler uma etiqueta numa foto. Cumpre WCAG 1.4.4
5	Cabeçalho sem <i>blur</i> <code>theme.css:331</code> → cor sólida a 96 %	5 min	B	Fim dos arrastos ao rolar em Android antigo
6	<i>Debounce</i> de 250 ms na pesquisa <code>history.js:213</code>	15 min	B	Escrever na pesquisa deixa de travar a app
7	Ecrã de arranque: 3 s → 600 ms	10 min	A	30 aberturas/dia × 2,4 s = ~1 min/dia devolvido

Vaga 1 — Menos Toques e Menos Peso (4 a 5 dias)

#	O quê	Tempo	Fonte	Efeito
8	Comprimir fotos com Canvas 1600 px lado maior, JPEG 0,75, no momento da captura	4 h	A B	Foto de 8 MB → ~300 KB. Salva o armazenamento, a sincronização e a memória de uma só vez. Cumpre o prometido no <code>PROJECT.md</code>
9	Botão de câmara na folha rápida ao lado do microfone	1 dia	A	Foto passa de 8 para 4 toques. Numa app de manutenção, a foto é o registo
10	Remover captura rápida duplicada do ecrã Hoje	2 h	A B	Acaba com os IDs duplicados e com a sobrecarga do ecrã Hoje. Um item resolve as duas revisões
11	Limpar ouvintes de cliques na página 3 sítios: <code>homeView</code> , <code>quickCapture</code> , <code>locationModal</code>	1 h	A B	App deixa de ficar lenta ao longo do turno
12	Vibração nas ações críticas <code>navigator.vibrate(40)</code> ao gravar, <code>[60,40,60]</code> ao eliminar	30 min	B	Confirmação que funciona ao lado de um gerador a trabalhar

13	Locais recentes no topo da lista	3 h	A B	Menos escrita entre 1 000+ salas. B também sugere pré-enchimento do último local
14	Componente Modal partilhado prende foco, Escape, <code>role="dialog"</code> , devolve foco	1 dia	A	Resolve 6 falhas de acessibilidade e substitui 8 implementações à mão
15	Substituir os 8 <code>confirm()</code> + <i>undo</i> ao eliminar	4 h	A B	Aspeto coerente e rede de segurança contra o toque errado. Usa o modal do item 14
16	Ligar a rota <code>metrics</code> ao menu Mais	15 min	A B	175 linhas já escritas e testadas passam a estar acessíveis

Vaga 2 — Arrumar a Casa (5 a 6 dias)

#	O quê	Tempo	Fonte	Efeito
17	Dividir <code>main.js</code> → <code>ui/reportForm.js</code> , <code>ui/settingsView.js</code> , <code>router.js</code>	1,5 dias	A B	1 176 linhas → ~150. Duas pessoas podem trabalhar sem conflitos no Git
18	Um só <code>esc()</code> em <code>src/utils/html.js</code>	3 h	A B	Remove 16 cópias e fecha um risco : a variante do <code>history.js</code> não escapa a apóstrofe
19	Componente <code>PrioritySelector</code> só classes CSS, sem cores escritas no elemento	4 h	A	Remove 4 cópias e faz o tema escuro funcionar nos botões de prioridade
20	Estilos inline → classes CSS ordem: <code>homeView</code> (62), <code>main</code> (55), <code>tasksView</code> (27), <code>quickCapture</code> (27)	2 dias	A B	Faz o tema escuro funcionar de verdade e permite tirar a maioria dos 78 <code>!important</code>
21	Contagem de tentativas por item + fila de presos usar o <code>retryCount</code> que já existe e ninguém lê	1 dia	A	A fila deixa de poder crescer sem fim. Definições mostram "3 registos não subiram"
22	Dividir <code>components.css</code> → <code>forms</code> , <code>cards</code> , <code>modals</code> , <code>navigation</code>	4 h	B	72 KB num ficheiro passam a quatro navegáveis. Faz-se <i>depois</i> do item 20, não antes
23	Reescrever ou apagar o <code>PROJECT.md</code>	2 h	A	Descreve ficheiros que não existem. Documentação que engana é pior que nenhuma
24	ESLint com duas regras próprias proibir <code>style="</code> e <code>font-size</code> em <code>.js</code>	3 h	B	Impede a recaída . Sem isto, os 220 inline voltam em seis meses

Vaga 3 — Aguentar Três Anos (3 a 4 dias)

#	O quê	Tempo	Fonte	Efeito
25	Fotos em tabela separada ligadas por <code>reportId</code> ; listas leem registos leves	1,5 dias	A	A mudança de estrutura que decide se a app aguenta . Torna a virtualização de listas desnecessária
26	Segredo partilhado na API + CORS restrito	4 h	A	Hoje qualquer pessoa com o endereço lê ou apaga tudo
27	Campo de técnico em cada registo ecrã de entrada com PIN	1 dia	A B	Valor legal do relatório municipal e pré-requisito do modo turno (item 31)

28	Parar a sincronização em segundo plano <code>visibilitychange</code>	2 h	A	Bateria: hoje sincroniza a cada 30 s mesmo com a app fechada
29	Lazy load de imagens <code>IntersectionObserver</code>	4 h	B	Com a compressão feita (item 8), passa de crítico a bom acabamento
30	Testar dropdowns em iPhone real	2 h	B	Suspeita plausível de <code>position:absolute</code> atrás do teclado. Medir antes de corrigir

5. Vaga 4 — O Que Faz o Técnico Preferir a App ao Caderno

As duas revisões chegaram às mesmas quatro funcionalidades. Esta é a parte que transforma um registo de avarias numa ferramenta de gestão do estádio.

A combinação que nenhuma das revisões viu isolada

QR + templates + histórico por equipamento formam um só fluxo. A revisão A pediu QR e histórico. A revisão B pediu templates e checklists. Juntos: o técnico lê a etiqueta do projetor → a app abre a ficha daquele projetor com o histórico → propõe o template "Manutenção de projetores" com os passos já escritos → ele marca as caixas e tira uma foto → fica registado no equipamento, não numa sala genérica. **Três toques, do bolso ao registo completo.** Nenhuma das partes vale metade do conjunto.

#	O quê	Tempo	Fonte	Porquê
31	Etiquetas QR + leitor pela câmara BarcodeDetector no Android; biblioteca leve no iOS. Ecrã que imprime folhas de etiquetas	1 semana	A B	A funcionalidade que mais muda o dia a dia. A coluna qr_code já existe em server/db.js:136 . Resolve os 8 toques e a busca em 1 000 salas ao mesmo tempo
32	Templates de avaria e checklists relvado, projetores, balneários, portões	3 dias	B	Escrever com luvas é o gargalo. Marcar caixas não é. Liga-se ao QR: cada equipamento sugere o seu template
33	Histórico e custo por equipamento timeline, total de horas, aviso ao 3.º arranjo no ano	3 dias	A B	Responde à pergunta que decide orçamentos: <i>"esta bomba já foi arranjada quatro vezes — vale a pena substituir?"</i>
34	Calendário de jogos e janelas de trabalho + lista de verificação do dia de jogo	4 dias	A B	Num estádio, <i>quando</i> se pode trabalhar é metade do problema. Não se corta a luz da bancada na véspera de um jogo
35	Modo turno / passagem de testemunho resumo do turno, notas de passagem, assinatura de quem recebe	3 dias	B	A melhor ideia funcional das duas revisões. "O projetor da torre norte faz um barulho estranho" é informação que hoje se perde entre turnos. Depende do item 27
36	Avisos de stock mínimo + desconto automático	2 dias	A	tools.min_qty e a tabela tool_moves já existem e estão vazias . Só falta usá-las
37	Data, hora e local gravados na foto	1 dia	A	Torna a foto autossuficiente como prova, mesmo depois de sair por WhatsApp. Usa o Canvas do item 8
38	Relatórios com assinatura, referência e brasão + exportação semanal/mensal automática	3 dias	A B	Um relatório que vai para a câmara municipal precisa de parecer oficial. Ambas as revisões pediram isto
39	Contratos e contactos de fornecedores elevadores, alarme de incêndio, climatização	2 dias	B	Saber a quem ligar e qual o contrato ativo poupa telefonemas numa emergência
40	Leituras de contadores água, eletricidade, gás — semanal	2 dias	A	Uma fuga na rega do relvado custa milhares antes de alguém a ver. Detetar cedo paga a app

6. Resultado Esperado

3 → 3 Toques para gravar avaria simples (já estava bom, mantém-se)	8 → 4 Toques para gravar com foto	6 → 2 Toques até ao histórico de um equipamento (via QR)
8 MB → 300 KB Peso de cada foto	1 176 → ~150 Linhas no <code>main.js</code>	220 → 0 Estilos inline no JS (com regra de linter)
16 → 1 Cópias da função <code>esc()</code>	-1 020 Linhas de código morto apagadas	120 → 120+ Testes a passar (nenhum se perde)

Ordem recomendada de execução

1. **Vaga 0 numa manhã.** Sete correções, quatro horas. Para as perdas de dados e faz a app deixar de mentir. Risco quase nulo.
2. **Vaga 1 na primeira semana.** A compressão de fotos (item 8) é a mais urgente das duas revisões — cada dia que passa são mais megabytes presos no telemóvel do técnico.
3. **Vaga 2 na segunda e terceira semanas.** Arrumar antes de construir. Se as funcionalidades da Vaga 4 entrarem antes desta arrumação, herdam a duplicação e o problema triplica.
4. **Vaga 3 quando houver dados a sério.** A separação das fotos (item 25) deve entrar antes de a base local passar os 500 registos.
5. **Vaga 4 por blocos, com o técnico ao lado.** Começar pelo QR + templates juntos — é onde a app deixa de ser um formulário e passa a ser uma ferramenta.

A regra que mantém isto de pé

O item 24 (ESLint que proíbe `style="` e `font-size` em ficheiros `.js`) parece o mais pequeno da lista e é o mais importante a longo prazo. Sem ele, os 220 estilos inline voltam em seis meses e esta revisão tem de ser feita outra vez. Com ele, a arrumação da Vaga 2 é definitiva.

O que fica igual — e é para ficar

JavaScript puro, sem estruturas externas. Offline-first como regra e não como modo. O `syncEngine.js` e a sua filosofia de nunca apagar sem confirmação. As datas em hora local do `tasksRepo.js`. Os tokens do `theme.css`. As transações atómicas dos repositórios. Os comentários em português que explicam o *porquê*. Nenhuma das duas revisões tocou nestas coisas — porque estão bem.

Consenso construído sobre duas revisões independentes · Todas as divergências resolvidas por execução real de comandos, não por concordância entre revisões · 5 afirmações da Revisão B corrigidas, 2 da Revisão A refinadas · 40 itens, ~7 semanas de trabalho no total, entregáveis em 5 vagas independentes · Estádio Municipal de Leiria · mmcrespo · 17 de agosto de 2026